数据结构：优先队列与堆
复习讲义

目录

1 优先队列 ADT (Priority Queue ADT) 2
1.1 模型 2

2 简单实现对比 (Simple Implementations) 2

3 二叉堆 (Binary Heap) 2
3.1 结构性质 (Structure Property) 2
3.2 数组表示 3
3.3 初始化 3
3.4 堆序性质 (Heap Order Property) 3

4 基本堆操作 (Basic Heap Operations) 4
4.1 插入：上滤 (Percolate Up) 4
4.2 删除最小值：下滤 (DeleteMin / Percolate Down) 4

5 其他堆操作 5
5.1 非最小关键字的查找 5
5.2 DecreaseKey 5
5.3 IncreaseKey 6
5.4 Delete 6
5.5 BuildHeap 6

6 优先队列应用 6
6.1 第 k 大元素 6

7 d-Heaps 7
7.1 定义 7
7.2 是否让 d 越大越好 7

8 全章总结 7

1 优先队列 ADT (Priority Queue ADT)

1.1 模型

优先队列 (Priority Queue) 支持删除优先级最高或最低的元素。课件以 min priority queue 为主，即每次删除最小元素。

对象：含零个或多个元素的有限有序表。

典型操作：

- `PriorityQueue Initialize(int MaxElements)`
- `void Insert(ElementType X, PriorityQueue H)`
- `ElementType DeleteMin(PriorityQueue H)`
- `ElementType FindMin(PriorityQueue H)`

2 简单实现对比 (Simple Implementations)

实现	插入	删除最小/最大
无序数组	加到末尾，$\Theta(1)$	扫描找目标，$\Theta(N)$；移除并移动元素，$O(N)$
无序链表	加到链表头，$\Theta(1)$	扫描找目标，$\Theta(N)$；已知位置后删除，$\Theta(1)$
有序数组	找插入位置并移动元素，$O(N)$	删除首/尾元素，$\Theta(1)$
有序链表	找插入位置，$O(N)$；链接结点，$\Theta(1)$	删除首/尾结点，$\Theta(1)$

课件指出：若删除次数不超过插入次数，有序结构在删除上有优势，但插入会变慢。使用平衡二叉搜索树可使插入和删除达到 $O(\log N)$，但常数和指针维护较重。优先队列更常用二叉堆实现。

3 二叉堆 (Binary Heap)

3.1 结构性质 (Structure Property)

完全二叉树：高度为 h、含 n 个结点的二叉树是 complete，当且仅当其结点与高度为 h 的满二叉树中从 1 到 n 编号的结点一一对应。

高度为 h 的完全二叉树结点数满足：

$$2^h \le n \le 2^{h+1} - 1,$$

因此：

$$h = \lfloor \log N \rfloor.$$

3.2 数组表示

二叉堆使用 1-based 数组表示，`BT[0]` 不使用：

- 根在下标 1。
- 下标 i 的父结点为 $\lfloor i/2 \rfloor$，若 $i = 1$ 则无父结点。
- 下标 i 的左孩子为 $2i$，若 $2i > n$ 则不存在。
- 下标 i 的右孩子为 $2i + 1$，若 $2i + 1 > n$ 则不存在。

顺序表示的核心引理

完全二叉树按层序编号后，父子下标可由简单整数运算得到。使用 1-based 下标能让父结点、左右孩子公式非常整齐。

3.3 初始化

课件的初始化会分配结构体和数组，并设置哨兵：

```
PriorityQueue Initialize( int MaxElements )
{
    PriorityQueue H;
    if ( MaxElements < MinPQSize )
        return Error( "Priority queue size is too small" );
    H = malloc( sizeof( struct HeapStruct ) );
    if ( H == NULL )
        return FatalError( "Out of space!!!" );
    H->Elements = malloc( (MaxElements + 1) * sizeof(ElementType) );
    if ( H->Elements == NULL )
        return FatalError( "Out of space!!!" );
    H->Capacity = MaxElements;
    H->Size = 0;
    H->Elements[0] = MinData;
    return H;
}
```

`Elements[0]` 是哨兵，插入时可简化上滤边界判断。

3.4 堆序性质 (Heap Order Property)

min tree：每个结点的关键字不大于其孩子的关键字。

min heap：既是完全二叉树，又是 min tree。

max heap：将堆序性质反过来，每个结点关键字不小于其孩子关键字。课件中左图是 max heap，右图是 min heap。

4 基本堆操作 (Basic Heap Operations)

4.1 插入：上滤 (Percolate Up)

新元素只能放在数组末尾，以保持完全二叉树结构。若新元素破坏堆序，就与父结点方向比较并上滤。

课件例子展示向堆中插入 21、17、9：

- 21 大于父结点，无需上滤。
- 17 小于父结点 20，上滤到合适位置。
- 9 可能一路上滤到根。

```
void Insert( ElementType X, PriorityQueue H )
{
    int i;
    if ( IsFull( H ) ) {
        Error( "Priority queue is full" );
        return;
    }

    for ( i = ++H->Size; H->Elements[i / 2] > X; i /= 2 )
        H->Elements[i] = H->Elements[i / 2];

    H->Elements[i] = X;
}
```

时间复杂度：

$$T(N) = O(\log N).$$

课件强调：上滤中使用移动赋值比反复 swap 更快。

4.2 删除最小值：下滤 (DeleteMin / Percolate Down)

`DeleteMin` 删除根结点。为保持完全二叉树，先取出最后一个结点作为候选，再从根开始下滤：

1. 保存根的最小值。
2. 用最后一个元素暂时填补根位置，并将堆大小减 1。
3. 每次选择较小孩子向上移动。
4. 当最后元素不大于较小孩子时，放入当前位置。

```
ElementType DeleteMin( PriorityQueue H )
{
    int i, Child;
    ElementType MinElement, LastElement;

    if ( IsEmpty( H ) ) {
        Error( "Priority queue is empty" );
        return H->Elements[0];
    }
    MinElement = H->Elements[1];
    LastElement = H->Elements[H->Size--];

    for ( i = 1; i * 2 <= H->Size; i = Child ) {
        Child = i * 2;
        if ( Child != H->Size &&
             H->Elements[Child + 1] < H->Elements[Child] )
            Child++;
        if ( LastElement > H->Elements[Child] )
            H->Elements[i] = H->Elements[Child];
        else
            break;
    }
    H->Elements[i] = LastElement;
    return MinElement;
}
```

时间复杂度：

$$T(N) = O(\log N).$$

课件提问：如果省略 `Child != H->Size` 条件会怎样？答案是可能访问不存在的右孩子。不能简单通过另一个哨兵完全替代这个判断，因为下滤需要知道左右孩子是否真实存在。

5 其他堆操作

5.1 非最小关键字的查找

堆只保证根是最小元素；除最小元素外，找任意关键字一般必须线性扫描整个堆。

5.2 DecreaseKey

`DecreaseKey(P, D, H)` 将位置 P 的关键字减少正数 D。关键字变小后可能比父结点小，因此需要上滤。

操作系统含义：提升某个进程优先级。

5.3 IncreaseKey

`IncreaseKey(P, D, H)` 将位置 P 的关键字增加正数 D。关键字变大后可能大于孩子，因此需要下滤。

操作系统含义：降低占用过多 CPU 的进程优先级。

5.4 Delete

`Delete(P, H)` 删除位置 P 的结点，可实现为：

```
DecreaseKey(P, infinity, H); DeleteMin(H);
```

思想是先把该元素变成堆中最小，使其上滤到根，再删除根。

5.5 BuildHeap

`BuildHeap` 将 N 个输入关键字放入空堆。若连续调用 N 次插入操作，复杂度为 $O(N \log N)$，课件指出这太慢。

更好的方法是：把所有元素先按数组顺序放好，然后从最后一个非叶结点开始，逆序执行 `PercolateDown`：

$$\left\lfloor \frac{N}{2} \right\rfloor, \left\lfloor \frac{N}{2} \right\rfloor - 1, \ldots, 1.$$

课件例子使用输入：

$$150, 80, 40, 30, 10, 70, 110, 100, 20, 90, 60, 50, 120, 140, 130$$

依次对位置 $7, 6, 5, 4, 3, 2, 1$ 下滤构造堆。

BuildHeap 复杂度

完美二叉树高度为 h 且含 $2^{h+1} - 1$ 个结点时，所有结点高度之和为

$$2^{h+1} - 1 - (h + 1).$$

因此自底向上的建堆总时间为

$$T(N) = O(N).$$

6 优先队列应用

6.1 k-th Largest Element（第 k 大元素）

课件应用问题：给定 N 个元素和整数 k，找第 k 大元素。

常见方法：

1. 排序后取第 k 大：$O(N \log N)$。

2. 建 max heap，执行 k 次 `DeleteMax`：$O(N + k \log N)$。

3. 维护大小为 k 的 min heap：扫描所有元素，堆顶保持当前第 k 大，复杂度 $O(N \log k)$。

4. 选择算法 Quickselect：平均 $O(N)$，但实现和最坏情况分析更复杂。

7 d-Heaps

7.1 定义

d-heap 是每个结点有 d 个孩子的堆。课件展示了 3-heap 的示意。
若采用 1-based 数组，对结点 i，可按层序推导父子下标。d 越大，树高越低：

$$h = O(\log_d N).$$

7.2 是否让 d 越大越好

课件答案是否定的，原因包括：

- `DeleteMin` 需要在 d 个孩子中找最小，需 $d - 1$ 次比较。
- 因此删除复杂度约为：
$$O(d \log_d N).$$
- 二叉堆中的乘 2、除 2 可由位移完成，而乘 d、除 d 不一定这么便宜。
- 当优先队列太大，无法完全放入主存时，d-heap 会更有意义，因为较大的分支因子可减少树高和外存访问次数。

8 全章总结

1. 优先队列支持按优先级删除元素，min heap 常用于实现 `DeleteMin`。

2. 简单数组/链表实现总会让插入或删除之一变慢。

3. 二叉堆依赖两个性质：完全二叉树结构性质和堆序性质。

4. 完全二叉树可用数组紧凑存储，父子下标关系是堆操作的基础。

5. `Insert` 使用上滤，`DeleteMin` 使用下滤，二者都是 $O(\log N)$。

6. 自底向上的 `BuildHeap` 是 $O(N)$，不是 $O(N \log N)$。

7. d-heap 在外存或大规模优先队列中有价值，但 d 不是越大越好。

“堆把树的形状固定住，再用局部上滤和下滤维护全局最小值。”